

Lecture 7:

Loop Clauses in OpenMP and Data Dependence

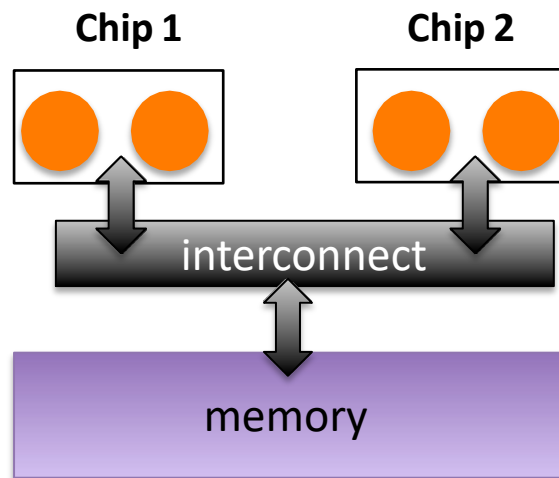
Beakal Gizachew Assefa

Today

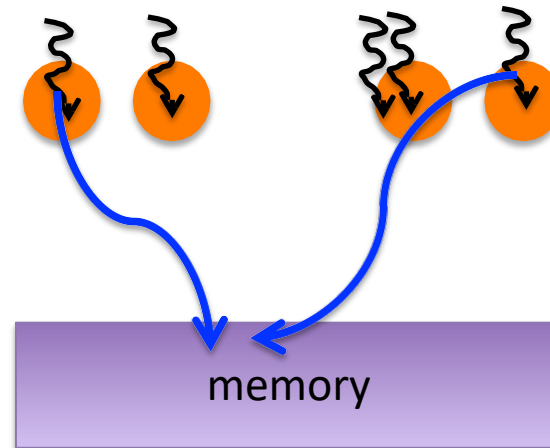
- More on OpenMP
 - Variable Scopes
 - Loop Scheduling
 - Other loop clauses
- Data Dependence
- Performance Tips
- Reading Chapter 5
- OpenMP Tutorial
 - <https://computing.llnl.gov/tutorials/openMP/>

Shared--Memory Programming Model

- More correct name: Shared--address space programming
 - Threads communicate through shared memory as opposed to messages
 - Threads coordinate through synchronization (also through shared memory).



Recall shared memory system
(can be either UMA, NUMA)



A Programmer's View of OpenMP

OpenMP will:

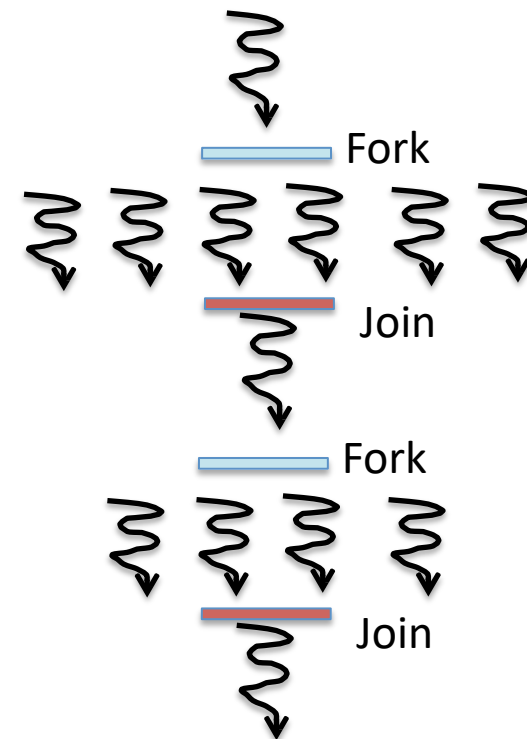
- Allow a programmer to separate a program into *serial regions* and *parallel regions*, rather than concurrently-executing threads.
- Hide stack management
- Provide synchronization constructs

OpenMP will not:

- Parallelize automatically
- Guarantee speedup
- Provide freedom from data races

OpenMP Execution Model

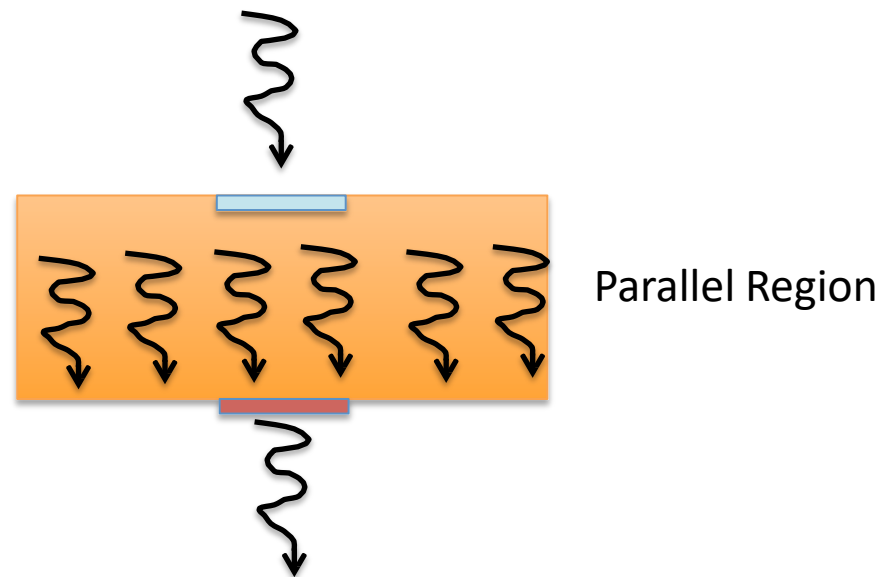
- Fork-join model of parallel execution
 - Begin execution as a single process (**master thread**)
- Start of a parallel construct:
 - Master thread creates team of threads (**worker threads**)
- Completion of a parallel construct:
 - Threads in the team synchronize – **implicit barrier**
- Only master thread continues execution



OpenMP Parallel Region

```
#pragma omp parallel [ clause [ clause ] ... ] new-line  
{  
    //structured block  
}
```

- Block of code to be executed by multiple threads in parallel
- Each thread executes the **same code redundantly (SPMD)**



Scope of Variables

- In serial programming, the scope of a variable consists of those parts of a program in which the variable can be used.
- In OpenMP, the scope of a variable refers to the set of threads that can access the variable in a parallel block.

Scope in OpenMP

- A variable that can be accessed by all the threads in the team has **shared** scope.
- A variable that can only be accessed by a single thread has **private** scope.
- The default scope for variables declared before a parallel block is **shared**.

Example:

```
#pragma omp parallel private(my_x, my_sum) shared(sum)
{ }
```

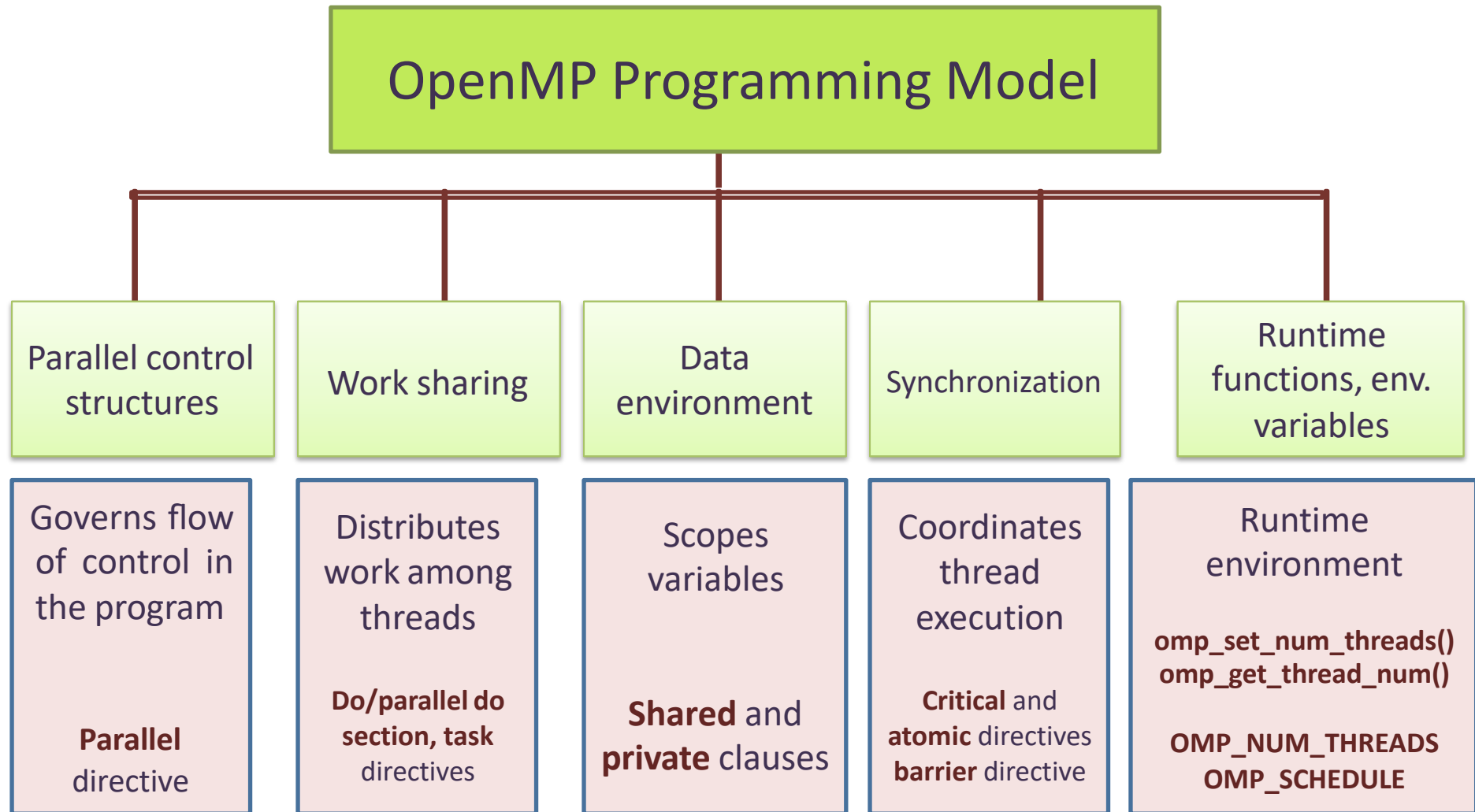

Default Clause

- Lets the programmer specify the scope of each variable in a block.

default(none)

- With this clause the compiler will require that we specify the scope of each variable we use in the block and that has been declared outside the block.

OpenMP Core Elements



More about OpenMP Loops

Parallel Loop Construct

- Compiler calculates loop bounds for each thread directly from *serial* source (computation decomposition)
- Compiler also manages data partitioning
- Implicit barrier at the end of the loop
- This assumes a parallel region has already been initiated, otherwise it executes in serial on a single processor.

Serial Program	Parallel Program	Parallel Program
<pre>double res[N]; for(int i=0; i < N; i++) do_huge_comp(res[i]);</pre>	<pre>double res[N]; #pragma omp parallel { #pragma omp for for(int i=0; i < N; i++) do_huge_comp(res[i]); }</pre>	<pre>double res[N]; #pragma omp parallel for for(int i=0; i < N; i++) do_huge_comp(res[i]);</pre>

Loop Scheduling

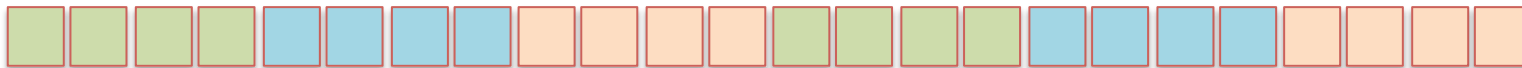
- schedule clause determines how loop iterations are divided among the thread team
 - **static([chunk])** divides iterations statically between threads
 - Each thread receives [chunk] iterations, rounding as necessary to account for all iterations
 - Default [chunk] is $\text{ceil}(\text{\# iterations} / \text{\# threads})$
 - **dynamic([chunk])** allocates [chunk] iterations per thread, allocating an additional [chunk] iterations when a thread finishes
 - Forms a logical work queue, consisting of all loop iterations
 - Default [chunk] is 1
 - **guided([chunk])** allocates dynamically, but [chunk] is exponentially reduced with each allocation

Loop Scheduling

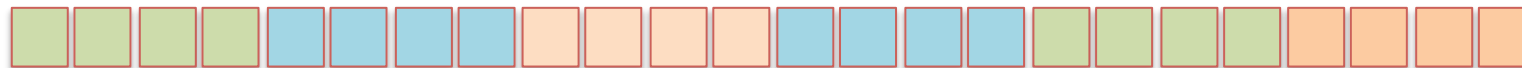
- Static, dynamic vs guided for 3 threads



`schedule(static, 4)`

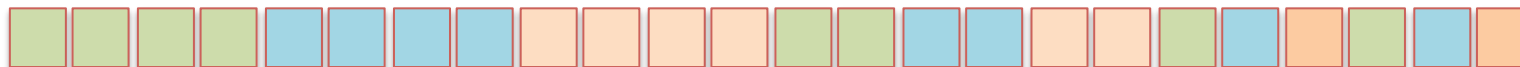


`schedule(dynamic, 4)`



Possible mapping

`schedule(guided, 4)`



Possible mapping

Impact of Loop Scheduling

- Load balance
 - Same work in each iteration?
 - Processors working at same speed?
- Scheduling overhead
 - Static decisions are cheap because they require no run-time coordination
 - Dynamic decisions have overhead that is impacted by complexity and frequency of decisions
- Data locality
 - Particularly within cache lines for small chunk sizes
 - Also impacts data reuse on same processor

Nowait Clause

- **NO WAIT / nowait:** If specified, then threads do not need to synchronize at the end of the parallel loop.

```
#pragma omp parallel
{
    #pragma omp for nowait
    for()
        . . .

    #pragma omp for
    for()
        . . .
}
```

When can we insert nowait?

There shouldn't be any data dependency across the loops

What is the advantage?

Removes serialization bottleneck

Do not assume the same thread will get the same iteration space across loops even if the loop ranges are the same

Collapse Clauses

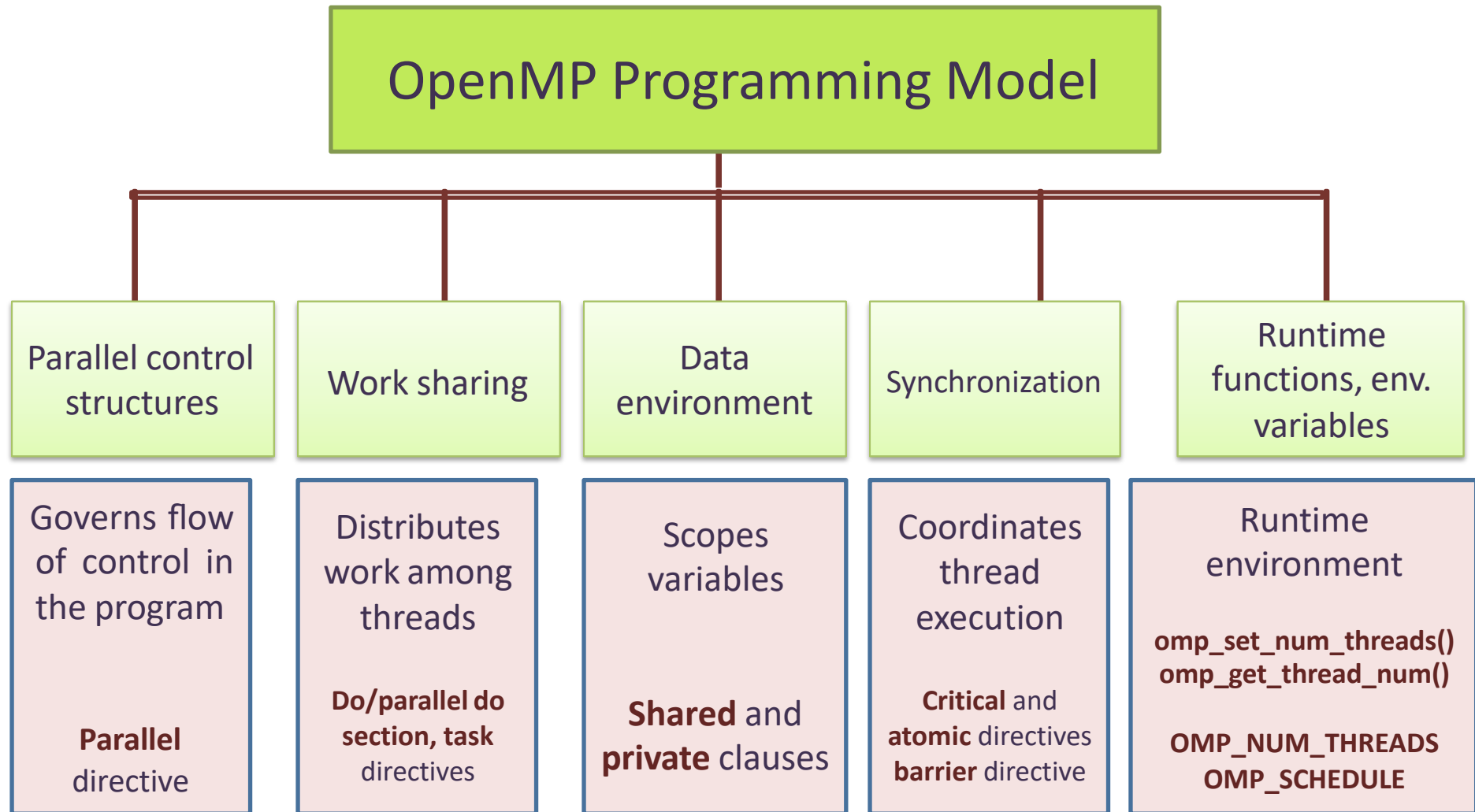
- **Parallel for loop only** applies to the outer most loop
- **collapse**: Specifies how many loops in a nested loop should be collapsed into one large iteration space

```
#pragma omp for  
for (i=0; i<N; i++)  
    for (j=1; j<N; j++)  
        . . .
```

```
#pragma omp for collapse(2)  
for (i=0; i<N; i++)  
    for (j=1; j<N; j++)  
        . . .
```

- In order to collapse a nested loop, there **shouldn't be** any **loop--carried dependence**

OpenMP Core Elements



Data Dependence

- A ***data dependence*** is an ordering on a pair of memory operations that must be preserved to maintain correctness.
- **Question:** When is parallelization guaranteed to be safe?
- **Answer:** If there are no data dependences across reordered computations.
- **Definition:** Two memory accesses are involved in a data dependence if they may refer to the same memory location and one of the accesses is a **write**.

Bernstein's conditions (1966)

- True Dependence –Read after Write (RAW)
 - If process P_i writes to a memory cell M_i , then no process P_j can read the cell M_i .
- Anti-Dependence –Write after Read (WAR)
 - If process P_i read from a memory cell M_i , then no process P_j can write to the cell M_i .
- Output Dependence –Write after Write (WAW)
 - If process P_i writes to a memory cell M_i , then no process P_j can write to the cell M_i .

Data Dependence of Scalar Variables

True (flow) dependence

a $=$
 $= a$

Definition: Data dependence exists from a reference instance i to i' iff

either i or i' is a write operation

i and i' refer to the same variable

i executes before i'

Anti-dependence

$= a$
 a $=$

Output dependence

a $=$
 a $=$

Input dependence (for locality)

$= a$
 $= a$

Actually, parallelizing compilers must formalize this to guarantee correct code.

Preserve Dependences

- Fundamental Theorem of Dependence:
 - Any reordering transformation that preserves every dependence in a program preserves the meaning of that program.
- Parallelization
 - Computations that execute in parallel between synchronization points are potentially reordered.
 - Is that reordering safe? According to our definition, it is safe if it preserves the dependences in the code.

```
#pragma omp barrier  
  
. . .  
#pragma omp for nowait  
for()  
. . .  
#pragma omp barrier
```

Data Dependence for Arrays

```
for (i=2; i<N; i++)  
    A[i] = A[i-2]+1;
```

Loop- Carried
dependence

```
for (i=1; i<=N; i++)  
    A[i] = A[i]+1;
```

Loop-Independent
dependence

- Recognizing parallel loops (intuitively)
 - Find data dependences in loop
 - No dependences crossing iteration boundary → parallelization of loop's iterations is safe

Loop-independent vs. – carried dependences

```
for (i=1; i<n; i++)  
{  
    S1: a[i] = a[i-1] + 1;  
    S2: b[i] = a[i];  
}  
  
for (i=1; i<n; i++)  
    for (j=1; j< n; j++)  
        S3: a[i][j] = a[i][j-1] + 1;  
  
for (i=1; i<n; i++)  
    for (j=1; j< n; j++)  
        S4: a[i][j] = a[i-1][j] + 1;
```

S1[i] → S1[i+1]: loop--carried
S1[i] → S2[i]: loop--independent

S3[i,j] → S3[i,j+1]:
• loop--carried on for j loop
• no loop-carried dependence
in for i loop

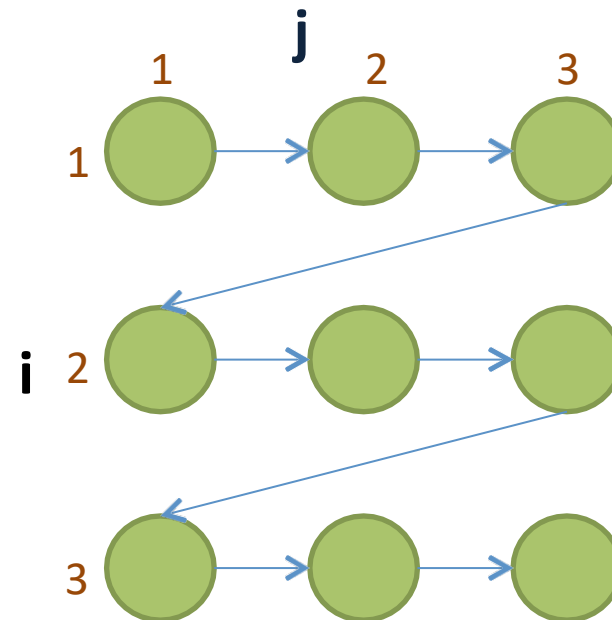
S4[i,j] → S4[i+1,j]:
• no loop-carried dependence in
for j loop
• loop--carried on for i loop

“Source: Yan Solihin, Fundamentals of Parallel Computer Architecture, 2008”.

Iteration--Space Traversal Graph

- Graphically the order of traversal in the iteration space
- A node represents a point in the iteration space
- A directed edge indicates the next point that will be encountered after the current point is traversed

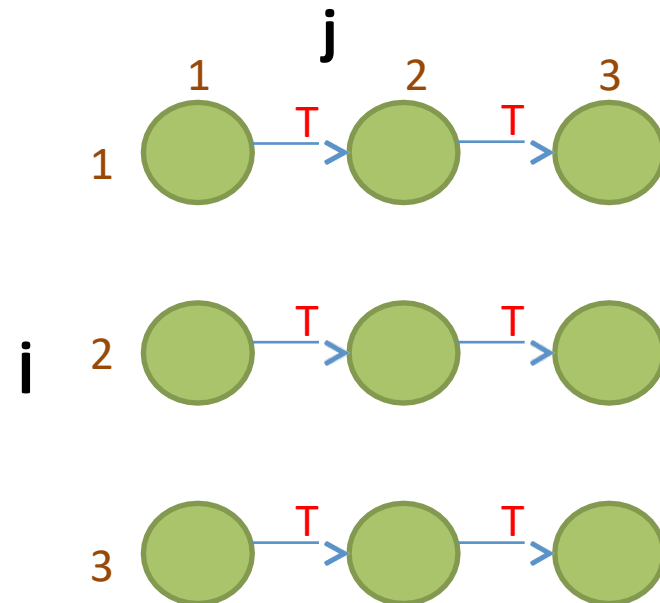
```
for (i=1; i<4; i++)  
  for (j=1; j< 4; j++)  
    S3: a[i][j] = a[i][j-1] + 1;
```



Loop-carried Dependence Graph

- Shows the true/anti/output dependence relationships graphically
- A node is a point in the iteration space
- A directed edge represents the dependence
- Dependent iterations cannot be parallelized

```
for (i=1; i<4; i++)  
  for (j=1; j< 4; j++)  
    S3: a[i][j] = a[i][j-1] + 1;
```



“Source: Yan Solihin, Fundamentals of Parallel Computer Architecture, 2008”.

OpenMP For

```
for (i=1; i<n; i++)
{
    S1: a[i] = a[i-1] + 1;
    S2: b[i] = a[i];
}
#pragma omp for
for (i=1; i<n; i++)

    for (j=1; j< n; j++)
        S3: a[i][j] = a[i][j-1] + 1;

for (i=1; i<n; i++)
#pragma omp for
    for (j=1; j< n; j++)
        S4: a[i][j] = a[i-1][j] + 1;
```

- For the first loop, S2 can be put in a separate loop in order for it to be parallelized

OpenMP compilers don't check for dependences among iterations in a loop.
It's programmer's responsibility.

For loop-Summary

Syntax:

```
#pragma omp for [ clause [ clause ] ... ] new-line  
for-loop
```

clause can be one of the following:

```
shared (list)
```

```
private( list)
```

```
reduction( operator: list)
```

```
schedule( type [ , chunk ] )
```

```
nowait
```

```
collapse( # )
```

Performance Tips

Tips are not only for OpenMP, for all shared memory programming models

- Number of Parallel Regions/Thread Creations
- First-touch policy
- False Sharing
 - Will be covered on a later lecture
- NUMA affinity mapping
 - Will be covered on a later lecture
- Note that these tips are not related to correctness of your program but should only affect the performance

Parallel Regions

```
#pragma omp parallel
{

}
#pragma omp parallel for
for (i=0; i<size; i++) {

}
#pragma omp parallel {

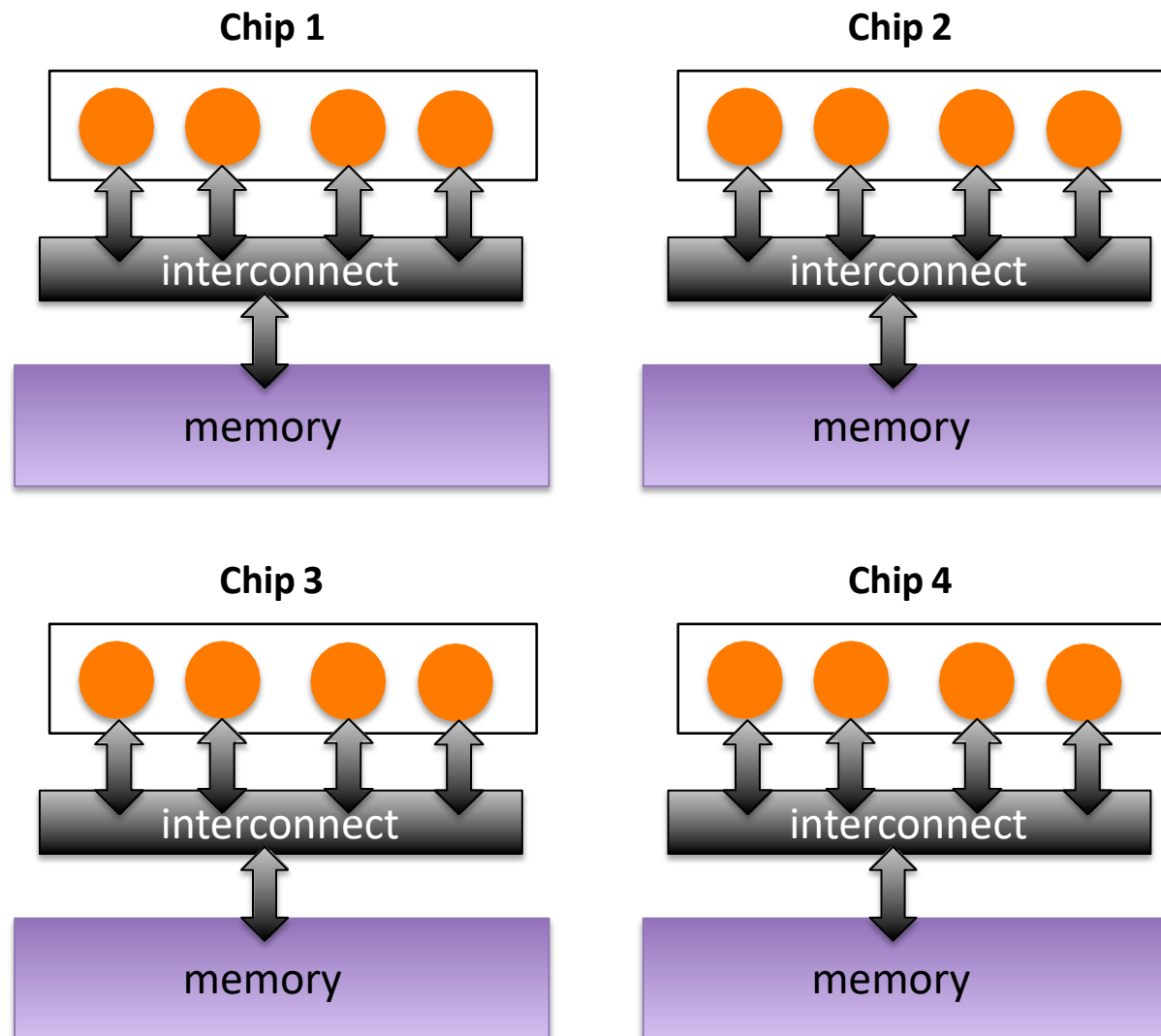
}
```

```
#pragma omp parallel
{

}
```

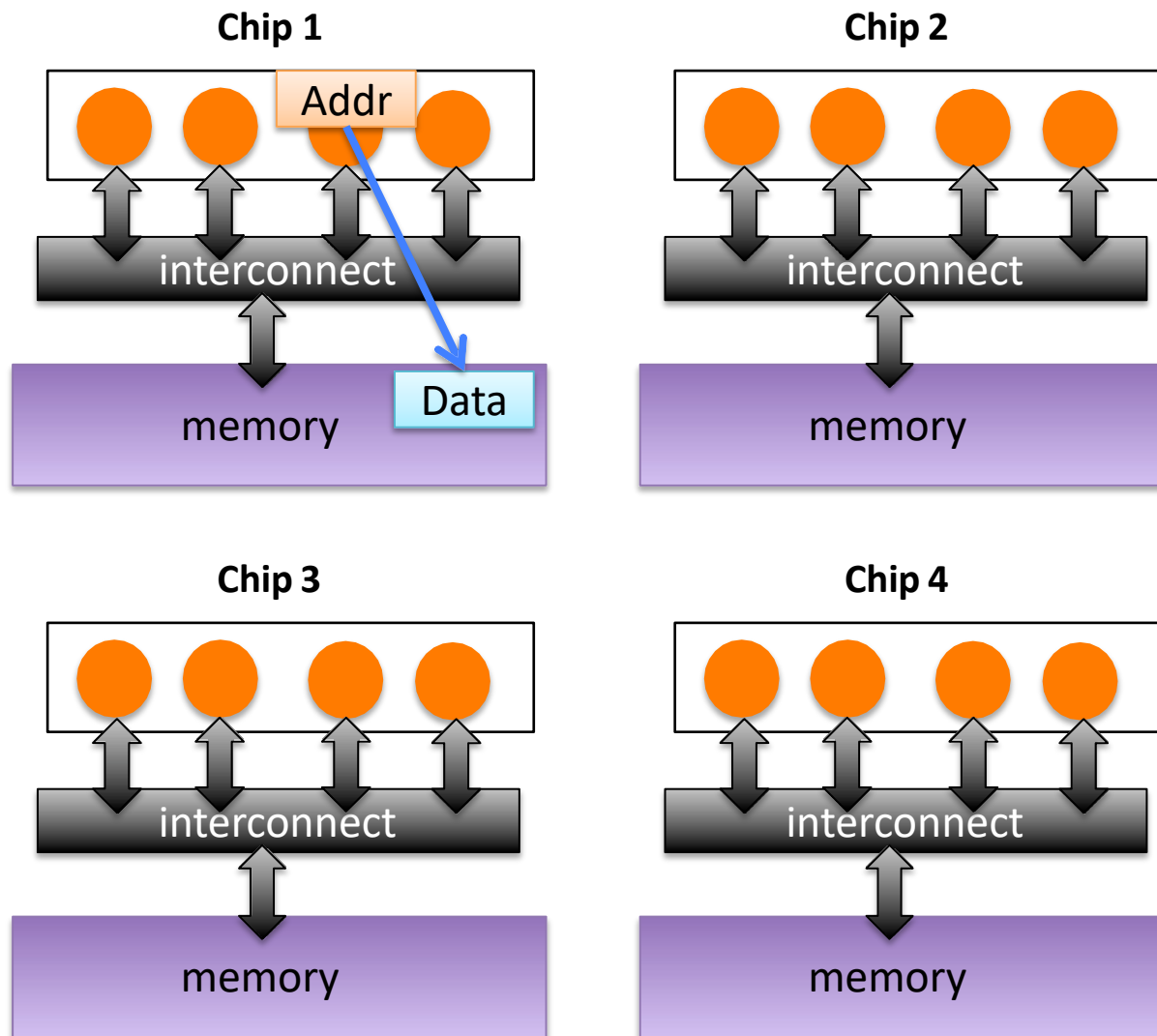
- Avoid creating too many parallel regions if you can
 - Each time, threads are created and joined!!!

NUMA



- 4 NUMA nodes
 - Four sockets

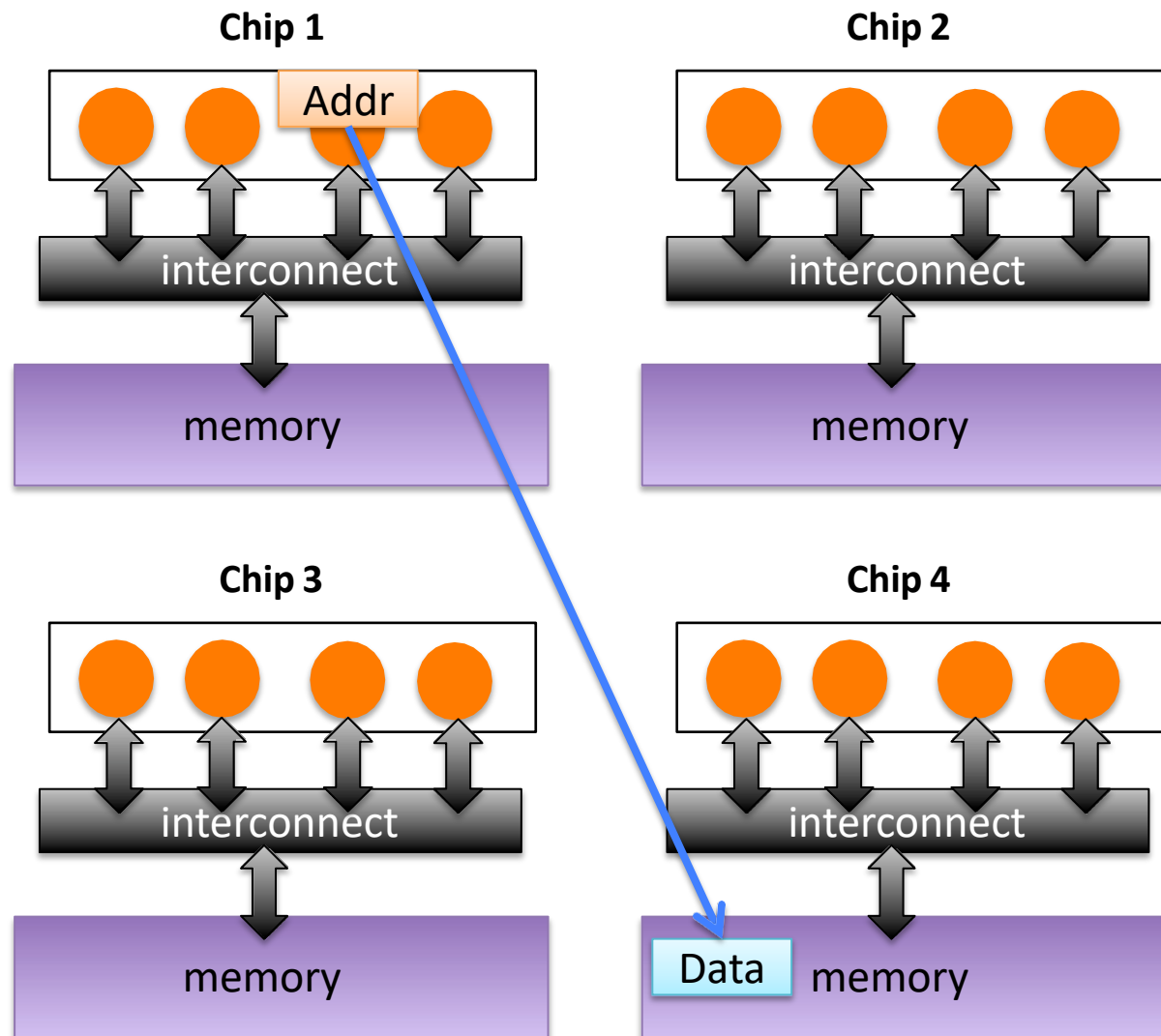
NUMA



- Accessing local memory is fast!!

- 4 NUMA nodes
 - Four sockets

NUMA

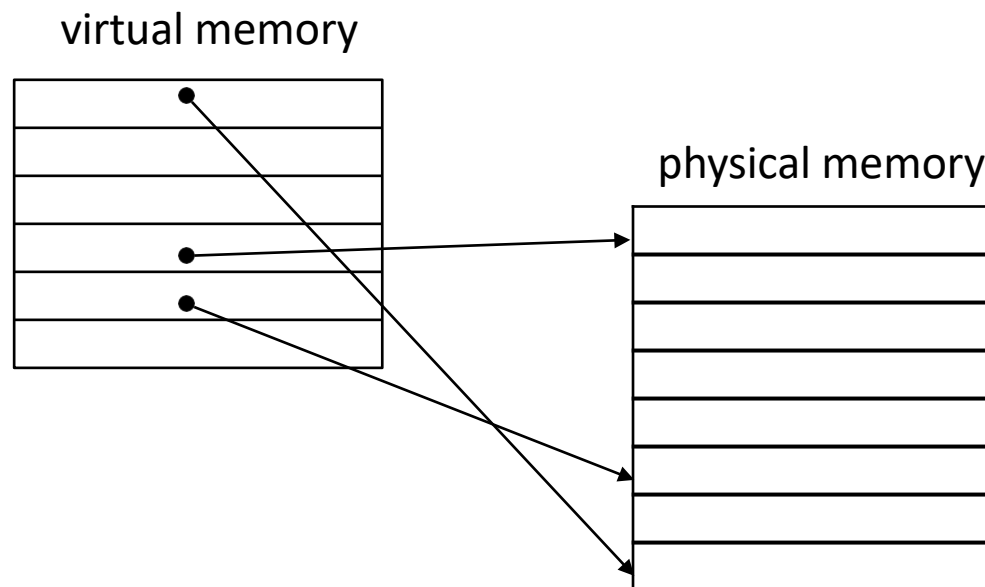


- Accessing remote memory is slow!!

- 4 NUMA nodes
 - Four sockets

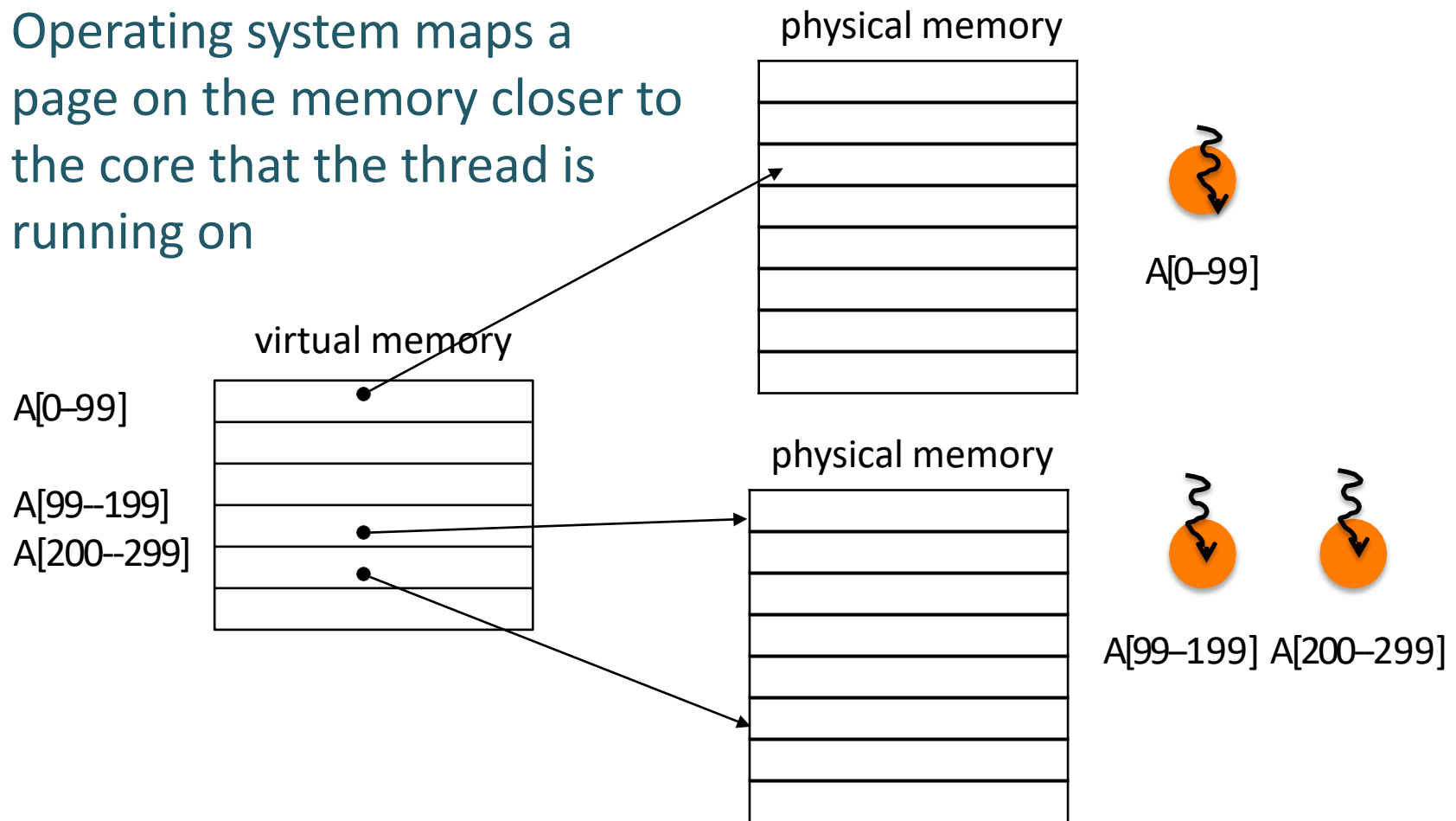
Memory Allocation

- Memory is allocated when it is first touched!
 - Call to malloc() only creates pages in virtual memory
 - Virtual memory to physical memory mapping is performed when a process/thread first touches to those pages



First Touch Policy on NUMA

- Operating system maps a page on the memory closer to the core that the thread is running on



Data Locality

- Master thread can allocate the memory
- But initialization should be done in parallel so that data locality is ensured
 - Data is close to the thread that uses it

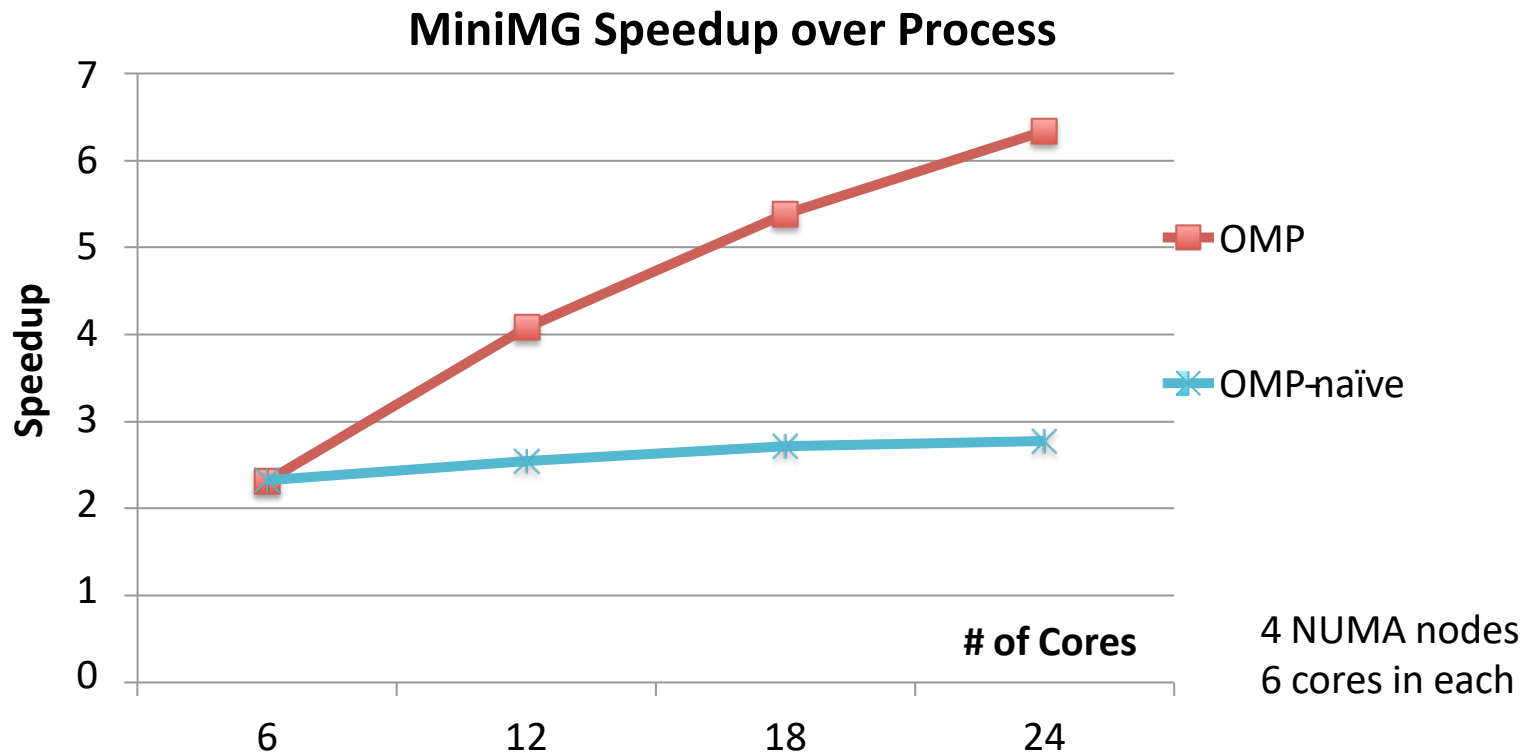
```
for (i=0; i<N; i++)  
    A[i] = 1.0;  
  
...  
  
#pragma omp parallel  
{  
    //calculation on A  
}
```

VS

```
#pragma omp parallel for  
for (i=0; i<N; i++)  
    A[i] = 1.0;  
  
...  
  
#pragma omp parallel  
{  
    //calculation on A  
}
```

Scalability

- Unless pages are dynamically migrated (by the Operating system or by the advanced programmer) the initial mapping does not change
- Not following first-touch policy can seriously inhibit scalability

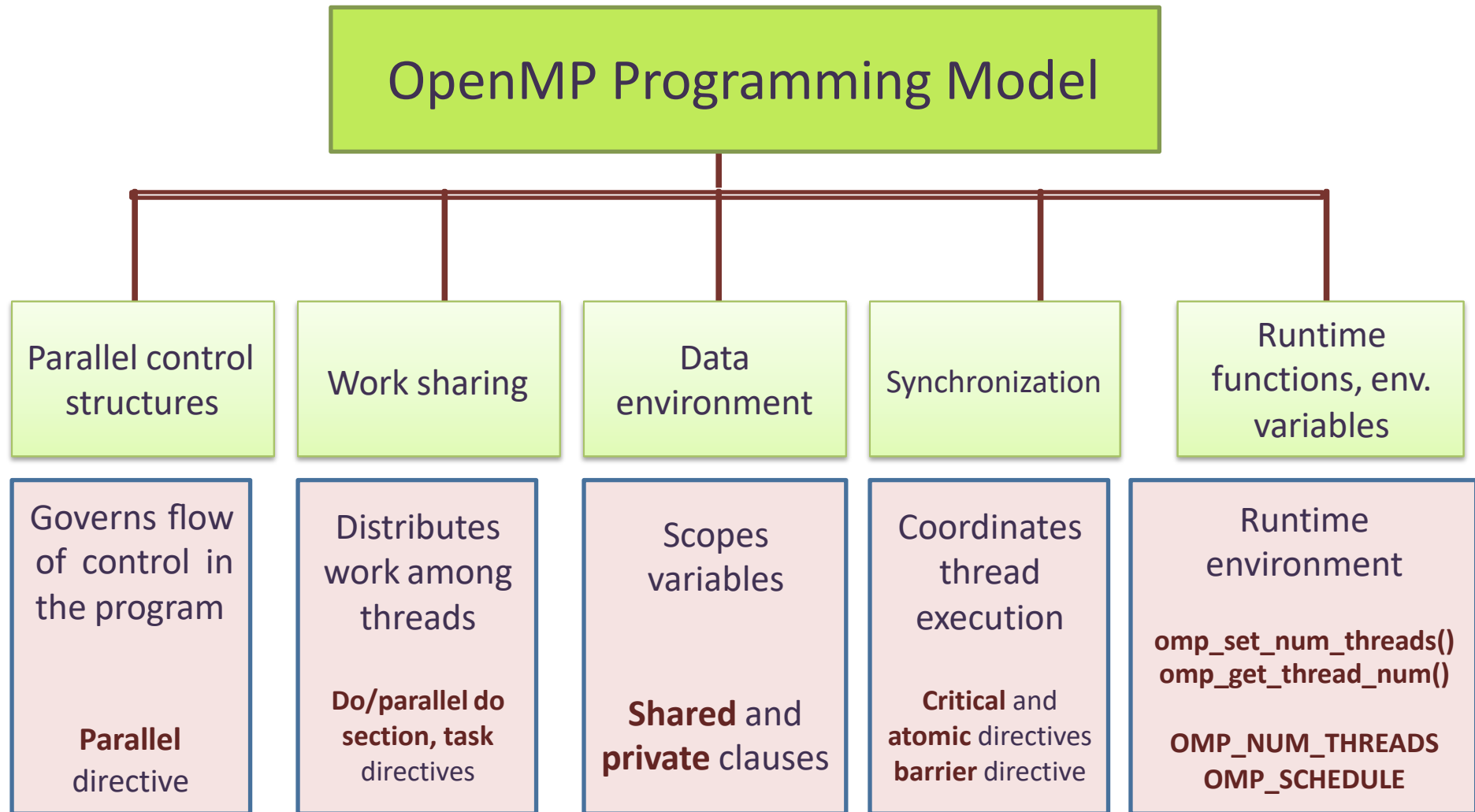


OpenMP Wall Time

- **OMP_GET_WTIME**
- Provides a portable wall clock timing routine
- Returns a double--precision floating point value equal to the number of elapsed seconds since some point in the past.

```
#include <omp.h>  
double omp_get_wtime(void)
```

OpenMP Core Elements



Acknowledgments

- These slides are inspired and partly adapted from
 - Didem Unat (Koç University)
 - Mary Hall (Univ. of Utah)
 - The course book (Pacheco)
 - Metin Aktulga (Machine State Univ.)
 - Yan Solihin, Fundamentals of Parallel Computer Architecture, 2008